

1.3.7 Helper Functions

Show Lecture Video

A great way to simplify coding is to use so-called **helper functions**. These are functions, like any other, but their purpose is to perform part of a computation for another function. For example:

```
1 def onesDigit(n):
2     return abs(n) % 10
3
4 def largerOnesDigit(x, y):
5     return max(onesDigit(x), onesDigit(y))
6
7 print(largerOnesDigit(134, 672)) # 4
8 print(largerOnesDigit(132, 674)) # Also 4
```

Run

Here, `onesDigit()` is a helper function for `largerOnesDigit()`. We could have written `largerOnesDigit()` without `onesDigit()`, but there are several key advantages to using helper functions:

1. Helper functions let us solve smaller problems at a time. This makes it much easier to reason over each logical chunk when solving larger problems.
2. Helper functions can be thoroughly tested with their own test functions. This isolates errors, which makes it much easier to test and debug our code.
3. Helper functions are reusable. We can use the same helper function for another function we may write in the future. This saves time and again reduces bugs.

In general, you should embrace using helper functions liberally. In fact, we will often require specific helper functions in the exercises to help you get used to this very effective approach.

Checkpoint 1

True or False: You can write multiple helper functions for one function.

☐ **False**

☐ **True**

Submit

Continue